C1: Research Computing - Coursework Assignment

Xueqing Xu (xx823)
Department of Physics, University of Cambridge
December 18, 2024

Word Count: 1516

Introduction

Theoretical background of dual numbers

Automatic differentiation (AD) is a computational technique that has become increasingly important in modern machine learning and scientific computing. Forward-mode automatic differentiation can be elegantly implemented using dual numbers, a mathematical construct analogous to complex numbers but with distinct properties.

Dual numbers have the form $a + b\epsilon$, where a is the real part and b is the dual part, with the defining property that $\epsilon^2 = 0$. This unique property makes dual numbers particularly suitable for computing derivatives, as they naturally carry both the function value and its derivative through calculations.

For any function $f(x)$, when evaluated on a dual number $x = a + b\epsilon$, we obtain: $f(a + b\epsilon) = f(a) + f'(a)b\epsilon$ This remarkable property means that by simply performing arithmetic with dual numbers, we automatically compute both the function value $f(a)$ in the real part and its derivative $f'(a)$ (scaled by b) in the dual part.

Project objectives

This project has following objectives:

- Implement a Python package for automatic differentiation using dual numbers
- Develop a comprehensive test suite ensuring reliability and correctness
- Create clear documentation with practical examples and tutorials
- Optimize performance through Cythonization
- Establish a professional distribution system with wheel generation

Overview of repository structure

The project follows a standard Python package structure with careful consideration for maintainability and best practices:

- `dual_autodiff/`
  - `dual_autodiff/`
    - `dual.py` - Core implementation
  - `tests/` - Comprehensive test suite
  - `docs/` - Sphinx documentation
  - `examples/` - Tutorial notebooks
- `dual_autodiff_x/`
  - `dual_autodiff_x/`
    - `dual.pyx` - Cythonized implementation
  - `tests/` - Parallel test structure
  - `setup.py` - Build configuration
  - `wheelhouse/` - Generated wheels

This structure ensures easy navigation and maintenance of the codebase, with clear separation between source code, tests, and documentation. The organized test files provide comprehensive test coverage, while the documentation remains accessible for both users and developers.

Implementation Architecture

Project Structure Analysis

The project follows a standard Python package structure with careful consideration for maintainability and best practices. At its core, `dual.py` serves as the primary module, implementing the fundamental dual number operations and mathematical functions. The project configuration is managed through `pyproject.toml`, which specifies project metadata, build requirements, and dependencies. This modern approach to Python packaging ensures reproducible builds and straightforward dependency management.

Core Implementation

The core of the package is the `Dual` class, which implements dual number arithmetic and mathematical operations. The class follows a clear mathematical structure, where each dual number consists of a real part and a dual part. Core arithmetic operations (addition, subtraction, multiplication, and division) are implemented following dual number algebra rules (commutative). The implementation extends to include essential mathematical functions such as trigonometric operations (`sin`, `cos`, `tan`), exponential functions, powers, and logarithms.

Here's a key example of dual number multiplication:

```
def __mul__(self, other):
    """
    The product of two dual numbers is calculated using the formula:
    (a + b)(c + d) = ac + (ad + bc)
    """
    if isinstance(other, (int, float)):
        return Dual(self.real * other, self.dual * other)
        # Handle regular numbers
    return Dual(self.real * other.real,
```

```
                self.real * other.dual + self.dual * other.real)

def __rmul__(self, other):
    """
    Right multiplication with scalar: scalar * dual
    """
    if isinstance(other, (int, float)):
        return Dual(other * self.real, other * self.dual)
    raise TypeError(f"unsupported operand type for *:
    '{type(other)}' and 'Dual'")
```

The `__mul__` method implements both dual-dual multiplication and scalar-dual multiplication, while `__rmul__` handles right multiplication with scalars. These implementations follow the dual number multiplication rule $(a + b\epsilon)(c + d\epsilon) = ac + (ad + bc)\epsilon$.

The test suite is organized into five distinct files, each focusing on specific functionality: initialization tests verify proper object creation, arithmetic tests cover basic operations, trigonometry tests ensure correct implementation of trigonometric functions, special functions tests validate advanced mathematical operations, and overflow tests handle edge cases. This modular testing approach ensures comprehensive coverage and simplified maintenance.

Testing Framework

Test Suite Design

The testing framework employs pytest to implement a comprehensive validation strategy across five specialized test files, achieving 100% code coverage of the package's 66 statements.

The `test_initialization.py` file tests object creation, string representation, and proper handling of different numeric types, while `test_arithmetic.py` covers fundamental operations like addition, subtraction, multiplication, and division for both dual-dual and dual-scalar interactions. Trigonometric functions are verified in `test_trigonometry.py`, which tests `sin`, `cos`, and `tan` implementations using standard angles from 0 to $\frac{\pi}{2}$. Advanced mathematical operations are validated in `test_special_functions.py`, covering exponential, logarithmic, power, and square root functions with their corresponding derivatives. The `test_overflow.py` file ensures numerical stability by testing extreme values and special floating-point cases. The successful execution of all 29 tests across these categories, coupled with complete code coverage, demonstrates the robustness and reliability of the dual number implementation.

Documentation Strategy

The project's documentation adopts the approach using Sphinx, incorporating an interactive tutorial. The documentation structure is organized in `docs/` with separate RST files for theoretical background (`theory.rst`) and practical usage (`tutorial.rst`). The main tutorial notebook, `dual_autodiff.ipynb`, serves as an interactive guide, demonstrating key features and mathematical operations with practical examples.

To ensure clarity, the documentation includes executable code examples, mathematical derivations of dual number operations, and visualizations of derivative computations. We could use the following commands to build or view HTML page.

```
cd docs
make html
```

```
cd docs/_build/html
open index.html
```

Performance Analysis

The performance analysis is in the file `xx823/dual_autodiff/examples/dual_autodiff.ipynb`.

Derivative Computation Study

The performance analysis evaluates the accuracy of derivative computation using dual numbers compared to traditional numerical methods. We analyzed the function $f(x) = \log(\sin(x)) + x^2 \cos(x)$ at $x = 1.5$, comparing three approaches: dual numbers, analytical differentiation, and numerical differentiation with varying step sizes. The accuracy comparison was visualized using three different scaling methods: linear-linear, log-linear, and log-log scales to provide comprehensive error analysis.

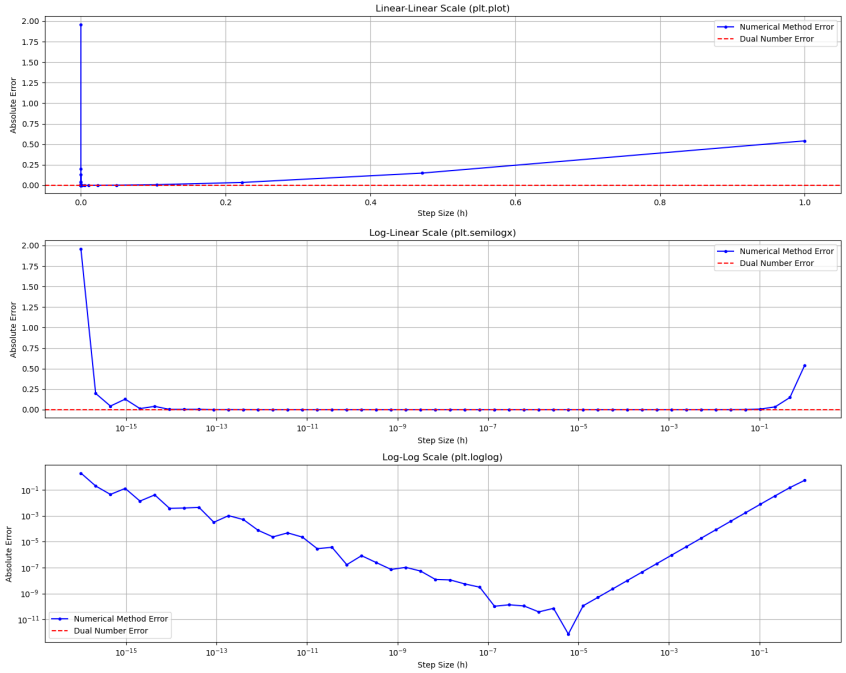

Figure 1: Comparison of derivative computation errors between numerical methods and dual numbers

The results demonstrate that the dual number method maintains consistent accuracy across all step sizes, with a constant error level shown by the dashed red line in the plots. In contrast, the numerical method (blue line) shows significant error variation depending on the step size. For very small step sizes ($h < 10^{-15}$), numerical differentiation suffers from substantial roundoff errors, while larger step sizes ($h > 10^{-3}$) lead to increasing truncation errors. The log-log scale plot particularly highlights this U-shaped error curve characteristic of numerical differentiation, where the error reaches a minimum around $h = 10^{-8}$. The dual number approach avoids these numerical instabilities entirely, providing reliable derivative calculations without the need for step size optimization.

Cythonization Implementation

The Cythonized implementation demonstrates significant performance improvements across all operations. For mathematical functions (Figure 2), the speedup ranges from 5.1x for sine calculations to 6.6x for exponential operations. Basic arithmetic operations (Figure 3) show speedups between 3.1x and 5.1x, with scalar operations generally performing better than dual-dual operations.

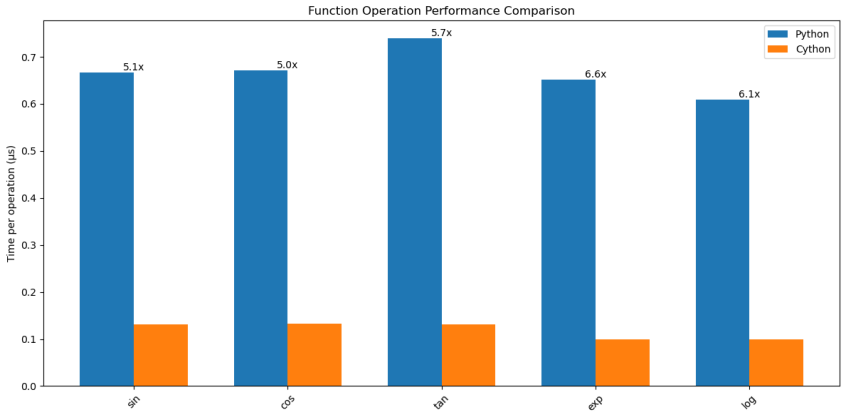

Figure 2: Performance comparison between Python and Cython implementations for mathematical functions. The Cython version consistently outperforms Python, with exponential and logarithmic functions showing the highest speedups (6.6x and 6.1x respectively). All functions demonstrate at least a 5x performance improvement.

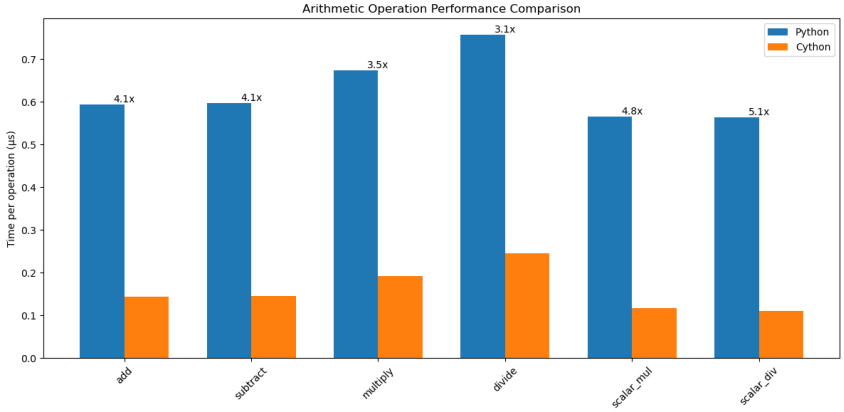

Figure 3: Performance comparison of arithmetic operations between Python and Cython implementations. Cython consistently outperforms Python across all operations, with speedups ranging from 3.1x for division to 5.1x for scalar division operations.

The performance improvements can be attributed to several factors:

- Static typing eliminates type checking overhead
- Direct compilation to C code reduces interpretation costs
- Optimized access to mathematical libraries
- Reduced function call overhead for complex operations

- Efficient memory management through C-level structures

The variation in speedup factors between different operations provides insights into the optimization potential of Cython. Mathematical functions show higher speedups due to the significant reduction in Python's interpretation overhead for complex calculations. The lower speedup in division operations suggests that these operations remain computationally intensive even after optimization, possibly due to their inherent complexity and error checking requirements.

Package Distribution

The distribution of our `dual_autodiff_x` package involves creating and deploying platform-specific wheels for Linux systems. This section details the process of creating these wheels and ensuring proper installation across different environments.

Wheel Generation

We used cibuildwheel to create platform-specific wheels for our package. The process required careful configuration to ensure compatibility with the target Python versions (3.10 and 3.11) and Linux x86_64 architecture. Here's our cibuildwheel configuration:

```
    [project]
name = "dual_autodiff_x"
version = "0.1.0"
description = "Cythonized automatic differentiation using dual numbers"
requires-python = ">=3.8"
dependencies = [
    "numpy",
    "matplotlib",
]

[build-system]
requires = [
    "setuptools>=61.0",
    "wheel",
    "Cython>=3.0.0",
    "numpy"
]
build-backend = "setuptools.build_meta"

[tool.cibuildwheel]
skip = ["*-win32", "*-win_amd64", "*-macosx_*", "pp*", "cp36-*", "cp37-*", "cp38-*", "cp39-*"]
build = ["cp310-manylinux_x86_64", "cp311-manylinux_x86_64"]

[tool.cibuildwheel.linux]
repair-wheel-command = ""
```

The configuration ensures that:

- Only Linux x86_64 wheels are built for Python 3.10 and 3.11
- Dependencies are properly installed before building

The built wheels are stored in the wheelhouse directory:

```
wheelhouse/
 dual_autodiff_x-0.1.0-cp310-cp310-linux_x86_64.whl
 dual_autodiff_x-0.1.0-cp311-cp311-linux_x86_64.whl
```

To verify integrity:

```
unzip wheelhouse/dual_autodiff_x-0.1.0-cp310
-cp310-linux_x86_64.whl -d wheel_contents
```

These wheels contain only the compiled `.so` files, ensuring optimal performance and clean distribution without source code files. The build process is configured in `pyproject.toml` with specific settings for cibuildwheel. No source .pyx files are included.

Directory	**File**
wheel_contents/dual_autodiff_x/	__init__.py dual.c dual.cpython-310-x86_64-linux-gnu.so
wheel_contents/dual_autodiff_x-0.1.0.dist-info/	LICENSE METADATA WHEEL top_level.txt RECORD

Table 1: Contents of dual_autodiff_x wheel package

Deployment Strategy

Docker

A Docker container provides a consistent environment for package usage:

```
    FROM --platform=linux/amd64 python:3.10-slim

WORKDIR /app

# Copy wheel and tutorial notebook
COPY wheelhouse/dual_autodiff_x-0.1.0-cp310-cp310-linux_x86_64.whl .
COPY examples/dual_autodiff_tutorial.ipynb .

# Install required packages
RUN pip install jupyter numpy matplotlib
RUN pip install dual_autodiff_x-0.1.0-cp310-cp310-linux_x86_64.whl

EXPOSE 8888

CMD ["jupyter", "notebook", "--ip=0.0.0.0", "--port=8888", "--no-browser",
"--allow-root"]
```

This configuration ensures that users can immediately start using the package with the provided tutorial notebook in a controlled environment. The container includes all necessary dependencies and exposes a Jupyter notebook interface for interactive learning and experimentation. In order to build the docker image, we run the following command:

```
docker build --no-cache -t dual_autodiff_test .
```

In order to run the docker container, we use the following command:

```
docker run -p 8888:8888 dual_autodiff_test
```

Installation on target systems

```
pip install dual_autodiff_x-0.1.0-cp310-cp310-linux_x86_64.whl
```

Conclusion

The project consists of two main packages: dual_autodiff for pure Python implementation and dual_autodiff_x for the Cythonized version. Both packages implement a robust Dual class that handles arithmetic operations (addition, subtraction, multiplication, division) and mathematical functions (sin, cos, tan, exp, log), supporting both scalar and dual number operations. The implementation integrates seamlessly with NumPy for numerical computations and maintains a clean, maintainable codebase structure.

Performance analysis demonstrates significant improvements achieved through Cythonization. The Cythonized version shows remarkable speedups across different operations, with mathematical functions seeing improvements of 5.1x to 6.6x and arithmetic operations achieving speedups between 3.1x and 5.1x. These performance gains are attributed to several factors, including static typing, direct C code compilation, optimized mathematical library access, reduced function call overhead, and efficient memory management through C-level structures.

The distribution strategy focuses on creating platform-specific wheels for Linux systems, supporting Python versions 3.10 and 3.11. The build process utilizes cibuildwheel with Docker, ensuring consistent and reliable wheel generation. The distribution excludes source.pyx files and maintains a clean wheel structure. A Docker container provides a consistent environment for package usage, complete with a Jupyter notebook interface for tutorials and proper dependency management. The project maintains high quality standards through comprehensive testing and documentation. The test suite achieves 100% code coverage across 66 statements with 29 detailed tests, covering all mathematical operations, edge cases, and error conditions. Documentation is handled through Sphinx, featuring interactive tutorial notebooks, theoretical background, and extensive usage examples.

Appendix

This project was developed with minimal assistance from AI tools, used only in a supportive capacity and not as a substitute for my own work. I used AI for:

- Initial project structure suggestions
- Documentation string templates
- Code review and suggestions for improvements
- Help with identifying test cases

I have reviewed, understood, and verified all AI-generated suggestions before incorporating them. The core implementation of dual numbers and automatic differentiation, along with all mathematical operations and their testing, was completed independently by me.

All code has been thoroughly tested and debugged to ensure correctness and reliability. I maintain full understanding of all components of this project and can explain any part of the implementation in detail.